

An Autonomous Eye-in-Hand Robotic System for Elevator Button Operation Based on Deep Recognition Network

Delong Zhu^{ID}, *Graduate Student Member, IEEE*, Zhe Min^{ID}, Tong Zhou^{ID}, Tingguang Li^{ID},
and Max Q.-H. Meng^{ID}, *Fellow, IEEE*

Abstract—Autonomous elevator button operation is an indispensable function for robot–elevator interaction, which has long been considered an intelligent solution for multifloor navigation of mobile robots. In this article, we present an autonomous robotic system with an eye-in-hand configuration to address the button operation problem. First, we develop a deep neural network for simultaneous button detection and character recognition, which provides accurate and robust perception inputs for the button operation system. Second, we present a button pose estimation algorithm with consideration of perception uncertainties, and the button pose is accurately estimated by fitting a least-uncertainty model. Based on the perception and pose estimation algorithms, a coarse-to-fine control schema is then proposed for driving the manipulator to accomplish the button operation task. Experimental results show that the proposed perception algorithm can outperform state-of-the-art methods in both recognition accuracy and running efficiency. The pose estimation and control schema also demonstrate their effectiveness in real-world button operation tasks. The data and code are available at our project webpage.

Index Terms—Autonomous robotic system, character recognition, elevator button detection, elevator button operation, multifloor navigation, pose estimation, visual servo control.

I. INTRODUCTION

ROBOT–ELEVATOR interaction is an indispensable function for mobile navigation across multiple floors in buildings. The ability of multifloor navigation can significantly enlarge the working space of mobile robots and enable them to deal with more application scenarios, such as package delivery, shopping guidance, and security patrol. Although some robotic systems have achieved such functions [1], [2], their interaction with elevators still relies on human assistance

or additional retrofits to the elevator, e.g., installing infrared devices [3]. Such robot–elevator interactions dramatically reduce the automation level of the robotic system and place much burden on human labor, which violates the principle of robot design that robots should emancipate human beings from hard work. Moreover, retrofitting elevators with infrared devices is costly, insecure, and maybe impossible if the robot needs to work in a large number of buildings. For instance, when a robot navigates to a previously unretrofitted elevator or the security key is not matched, it will lose the ability to interact with the elevator. Therefore, it is necessary to develop an intelligent solution that enables the robot to interact with elevators independently and without any external auxiliary or retrofit to elevators.

However, the intelligent robot–elevator interaction is a very challenging task, the implementation of which depends on the successful integration of multiple components, e.g., button operation, mobile navigation, and motion planning. In this article, we focus on the task of button operation and aim to enable the robot to perceive and operate elevator buttons in a fully autonomous manner. To achieve this purpose, we investigate three fundamental problems: button recognition, button pose estimation, and button operation control.

Button recognition refers to the task of button detection and character recognition, which is performed on elevator panel images and aims to locate the target button. Conventional methods [4]–[10] leveraging low-level image features, e.g., image densities, points, and gradients, are sensitive to light changes and have limited robustness. Recent studies [11]–[14] are typically based on deep learning techniques and exhibit more robust performance, but their accuracy remains to be improved due to the severe data imbalance problem of elevator panel data sets. Since the number of low-rise buildings is always larger than that of high-rise ones, these data sets are dominated by categories of small floor numbers, which can easily overfit the button recognition model. Moreover, most recognition algorithms [6], [9], [11]–[14] are developed separately from the button operation process and thus neglect the importance of runtime efficiency, which indeed determines the control frequency and the final operation accuracy.

To tackle these problems, we develop a deep neural network, called OCR-RCNN-V2, based on our previous OCR-RCNN [14]. Three major improvements are made: 1) button candidates used for character recognition are replaced with more

Manuscript received October 13, 2020; revised November 17, 2020; accepted November 22, 2020. Date of publication December 15, 2020; date of current version December 24, 2020. This work was supported in part by Hong Kong RGC GRF under Grant #14200618 and in part by Hong Kong ITC ITSP Tier 2 under Grant #ITS/105/18FP. The Associate Editor coordinating the review process was Yan Zhuang. (Corresponding author: Max Q.-H. Meng.)

Delong Zhu, Zhe Min, Tong Zhou, and Tingguang Li are with the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: zhudelong@link.cuhk.edu.hk; zhemin@link.cuhk.edu.hk; tongzhou@link.cuhk.edu.hk; tgli@link.cuhk.edu.hk).

Max Q.-H. Meng is with the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong, also with the Department of Electronic and Electrical Engineering, Southern University of Science and Technology, Shenzhen 518055, China, and also with the Shenzhen Research Institute, The Chinese University of Hong Kong, Hong Kong, on leave from the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: max.meng@ieee.org).

Digital Object Identifier 10.1109/TIM.2020.3043118

accurate ones predicted by the detection branch of OCR-RCNN; 2) a lightweight feature extractor is incorporated into the OCR branch to improve the recognition accuracy; and 3) the backbone network of OCR-RCNN is simplified into a more efficient implementation for reducing the network scale. Based on these improvements, OCR-RCNN-V2 achieves four times the running speed of OCR-RCNN and increases the recognition accuracy by 12%.

Button pose estimation aims to estimate the rigid transformation, including rotation and translation, of elevator buttons to the manipulator. In typical manipulation systems, the transformation between objects and the base frame of the manipulator is fixed and thus can be precalibrated accurately. However, for button operation, the transformation needs to be estimated from sensor measurements. In this work, an eye-in-hand configuration is adopted for the button operation system, in which an RGBD camera is mounted on the manipulator, and the captured images are used for pose estimation. To improve the pose estimation accuracy, we propose an uncertainty model to fit the noise introduced by inaccurate sensor measurements, based on which the button pose estimation is formulated as a maximum likelihood estimation (MLE) problem. Through optimizing the likelihood function, an accurate button pose with the least uncertainty is obtained.

Compared with button recognition and pose estimation, operation control is a well-developed research area, and there are many off-the-shelf algorithms in the literature [15]–[18]. In this work, we propose a coarse-to-fine control schema that integrates an efficient position controller [19] and an accurate visual servo controller [16] to address the control problem. The design of the control schema follows two principles: easy to implement and meeting the accuracy requirement. During the execution phase, the position controller first drives the manipulator to the desired location, and then the visual servo controller is launched to fine-tune the pose of the manipulator. Herein, limited to the accuracy of pose estimation and hand-eye calibration, the position controller cannot finish the operation task independently. Therefore, we integrate a servo controller, which performs pixel-level control, to improve the control accuracy. By combining the advantages of the two controllers, the control problem is effectively addressed.

The contributions of this article are summarized as follows.

- 1) We develop a novel button recognition network, OCR-RCNN-V2, which outperforms state-of-the-art methods in recognition accuracy and running efficiency.
- 2) We propose an efficient uncertainty model to tackle button pose estimation problem, which considerably improves the accuracy of button pose estimation.
- 3) Based on the button recognition and pose estimation algorithms, we propose a coarse-to-fine control schema, which can effectively conduct button operation tasks.
- 4) We integrate the modules to an eye-in-hand robotic system, which for the first time gives a systematic solution for the elevator button operation problem.

The remainder of this article is organized as follows. Section II briefly surveys recent studies on elevator button operation. Sections III and IV detail the proposed system and

algorithms. Experiment results are presented in Section V, and we conclude this article in Section VI.

II. RELATED WORK

A. Conventional Methods

In early studies, researchers have attempted to build autonomous robotic systems for elevator button operation. For instance, in the 2002 AAAI Mobile Robot Challenge [20] and the 2008 TechX Challenge [21], one of the key missions was to navigate the robot to different floors by operating elevator buttons. Even with the button images provided beforehand, the difficulty of autonomously operating an elevator was still outstanding. After that, Klingbeil *et al.* [4] presented an elevator button operation system that can work in previously unknown elevators. The authors designed an image processing pipeline to tackle the button recognition problem and achieve high accuracy on a data set with 50 test images. However, this work has strong assumptions that all the images are in favorable conditions and without any distortion. When performing button operation, the robot is assumed to be already at the desired pose, and only a simple pressing action is needed. The pose estimation and control algorithm are also not mentioned there. Kang *et al.* [5] presented a mobile navigation system that allows the robot to autonomously get on/off elevators. A template matching algorithm is developed to help recognize floor numbers, but no solutions are presented for button operation. Kim *et al.* [6] also proposed a button recognition algorithm based on template matching. By applying a homography-based image transform to the matching process, the recognition algorithm shows better robustness to specular reflections and perspective distortions. Wan *et al.* [7] and Abdulla *et al.* [8] proposed to leverage low-level image features, e.g., edge information, to detect elevator buttons first and then perform character recognition to locate the target button. Similar to [4]–[6], the two methods are developed on nonrobust image features and thus sensitive to light changes.

B. Deep Learning Based Methods

In recent years, deep learning techniques have been extensively applied in traditional automation problems [22]. Visual object recognition for manipulation tasks becomes a more promising research area in robotics. As a specific application, elevator button operation also receives much attention recently. Dong *et al.* [11] proposed a hybrid button recognition framework, in which the edge information is used to generate button candidates, and a classification neural network is leveraged to label the candidates. Yang *et al.* [12] further developed the idea mentioned in [11] and utilized a multiobject detection framework, called YOLO [23], to perform end-to-end button recognition. When performing button operation, a similar strategy with [4] is adopted, but the desired pose is achieved by tuning the mobile base. Liu and Tian [13] also adopt a detection network, called SSD [24], to recognize elevator buttons. The major difference with the previous work is that a segmentation network is additionally integrated to highlight operable regions, which are very useful for tackling some specially designed buttons. The methods presented in [12]

and [13] achieve more robust performance than [11], but due to the lack of large-scale data sets, their accuracies are not sufficiently evaluated. To address this problem, Zhu *et al.* [14] collected a large-scale data set and conduct extensively comparison experiments, based on which they find that multiobject detection models are easily overfitted due to the severe data imbalance problem. Therefore, Ren *et al.* [25] integrate a character recognition branch into the faster-RCNN framework. In this way, the rare button categories, usually large numbers, are recognized by characters, which significantly ease the data imbalance problem. However, the recognition accuracy of this work is not satisfying, and the button operation procedure is also not mentioned. Wang *et al.* [26] presented a specially designed manipulator for button operation, which achieves high operation accuracy, but the button recognition and pose estimation are accomplished by using fiducial markers rather than in an autonomous manner.

C. Unsolved Problems

As we can see, existing studies typically focus solely on the button recognition problem and assume that the robot pose is well-tuned for button operation. Such settings yield several problems: 1) the assumption is not realistic and hard to fulfill in real-world applications; 2) the efficiency of button recognition is not emphasized; and 3) the important button pose estimation and control problems are not addressed. Motivated by these issues, we propose a systematic solution to address the elevator button operation problem.

III. SYSTEM OVERVIEW

As shown in Fig. 1, the proposed button operation system is with an eye-in-hand configuration, and the software architecture includes three components: the recognition thread, the pose estimation thread, and the execution thread. In the recognition thread, we perform button detection and character recognition to locate the target button. In the pose estimation thread, the target button is tracked and further processed by a robust line detector to help locate button corners, based on which we sample button point clouds and conduct pose estimation. In the execution thread, the estimated pose and button corners are used to formulate control errors for the position controller and visual servo controller, respectively. The designed system is based on the following assumptions: 1) the button should be rectangular,¹ as the current visual servo controller is based on corner points; 2) the button panel has been captured by at least one of the RGBD sensors [Fig. 1(a)]; and 3) there is no collision with other objects during the button operation process. The last two assumptions can be fulfilled by robotic exploration [27], [28] and MoveIt framework [29].

IV. ELEVATOR BUTTON OPERATION

Throughout this section, the following notations are used.

- 1) $\mathbf{f} \in \mathbb{R}^{N \times D}$: The feature map of a button candidate (Fig. 2), where N and D are feature length and channel depth, respectively. $\mathbf{f}_i \in \mathbb{R}^{D \times 1}$, $1 \leq i \leq N$.

¹For circular buttons, a solution is also provided at the end of this article.

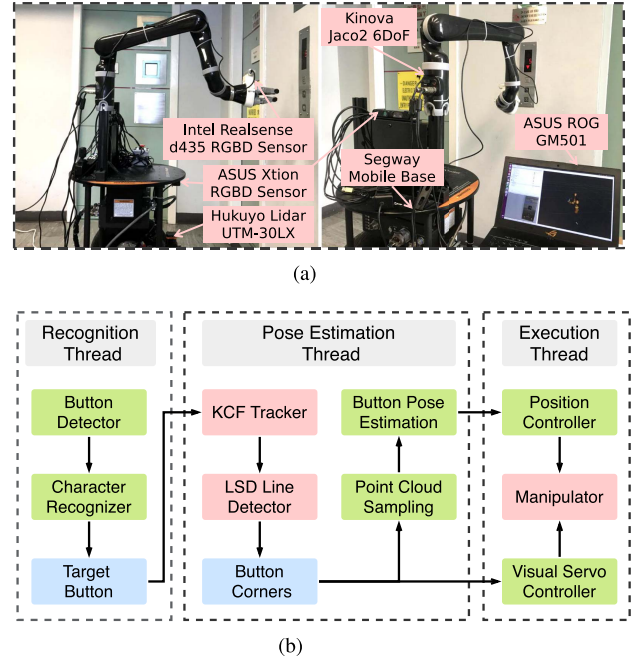


Fig. 1. Overview of the elevator button operation system. The contributed components are indicated by green blocks. (a) Hardware settings of the system. (b) Software architecture of the system.

- 2) $\alpha \in \mathbb{R}^{N \times 1}$: The attention vector used to highlight spatial information of the characters on a button.
- 3) $\mathbf{o}, \mathbf{g} \in \mathbb{R}^{L \times 1}$: The predicted and ground-truth character labels with one-hot encoding, respectively, where L is the number of character categories.
- 4) $\mathbf{x}, \mathbf{y}, \mathbf{h} \in \mathbb{R}^{C \times 1}$: The input, output, and hidden states of RNN, respectively, where C is the number of RNN cells.
- 5) $\mathbf{W}_c, \mathbf{W}_f \in \mathbb{R}^{C \times D}, \mathbf{W}_g \in \mathbb{R}^{C \times L}, \mathbf{W}_h \in \mathbb{R}^{C \times C}, \mathbf{W}_y \in \mathbb{R}^{L \times C}, \mathbf{W}_o \in \mathbb{R}^{L \times D}, \mathbf{V} \in \mathbb{R}^{C \times 1}$: The trainable parameters of character recognizer [Fig. 2(b)].
- 6) $\mathbf{x}, \mathbf{y} \in \mathbb{Z}^{2 \times 1}$: The pixel coordinate of a feature point in image plane, represented by $[u, v]^T$.
- 7) $\bar{\mathbf{x}}, \bar{\mathbf{y}} \in \mathbb{Z}^{3 \times 1}$: The homogeneous coordinates of $\mathbf{x}$ and $\mathbf{y}$, represented by $[\mathbf{x}^T, 1]^T$ and $[\mathbf{y}^T, 1]^T$, respectively.
- 8) $\mathbf{p}, \mathbf{q} \in \mathbb{R}^{3 \times 1}$: The coordinate of a feature point in a certain 3-D frame, represented by $[x, y, z]^T$.
- 9) $\mathbf{K}$: The intrinsic matrix of a camera.
- 10) $\Sigma = \text{diag}([\sigma_x, \sigma_y, \sigma_y])$: The diagonal covariance matrix of the uncertainty model, $\sigma = [\sigma_x, \sigma_y, \sigma_y]^T$.
- 11) $T = [R(\theta)|\mathbf{t}]$: The transformation between two frames, where θ is the angle-axis representation of rotation, and $R(\theta)$ is the corresponding matrix representation.

A. Button Recognition

1) *Network Architecture*: Based on our previous work [14], i.e., the end-to-end OCR-RCNN [Fig. 2(a)], we propose a new architecture, called OCR-RCNN-V2 [Fig. 2(b)], which separates the button recognition task into two subnetworks, i.e., the button detection network and the character recognition network. The design of the detection network follows a typical Faster-RCNN architecture [25] and comprises of a bounding

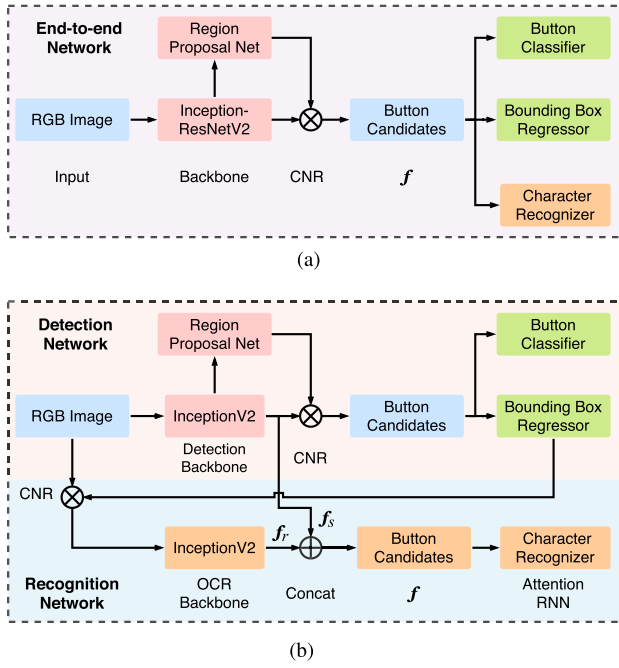


Fig. 2. Architectures of OCR-RCNN and OCR-RCNN-V2. CNR is the crop-and-resize operation. Concat is short for concatenation. (a) End-to-end architecture of OCR-RCNN. (b) Two-step architecture of OCR-RCNN-V2.

box regressor and a binary button classifier. The major difference is that the button candidates used for character recognition are generated by the bounding box regressor rather than the region proposal network adopted by OCR-RCNN. The regressor can generate more accurate button candidates, which are crucial for improving character recognition accuracy, as we shall see in the following discussions about the recognition process.

The character recognizer is essentially an attention-RNN [30]. By taking $\mathbf{h}^0 = \mathbf{0}$ and $\mathbf{g}^0 = \mathbf{0}$, the RNN decoding process, i.e., the character recognition, can be expressed as follows:

$$(\mathbf{y}^t, \mathbf{h}^t) = \text{RNNstep}(\mathbf{x}^t, \mathbf{h}^{t-1}) \quad (1)$$

$$\mathbf{x}^t = W_g \mathbf{g}^{t-1} + W_c \sum_i \alpha_i^{t-1} \mathbf{f}_i \quad (2)$$

$$\alpha_i^t = \text{softmax}(V^T \tanh(W_h \mathbf{h}^t + W_f \mathbf{f}_i)). \quad (3)$$

Based on the outputs of attention-RNN, the characters on a button are predicted as follows:

$$\mathbf{o}^t = \text{softmax}\left(W_y \mathbf{y}^t + W_o \sum_i \alpha_i^t \mathbf{f}_i\right). \quad (4)$$

Note that the ground-truth label $\mathbf{g}$ in (2) should be replaced with the predicted one in the inference phase. The attention α is leveraged to highlight spatial information of the characters for guiding the RNN to sequentially recognize them. According to the decoding process, we can see that the performance of character recognition highly depends on $\mathbf{f}$. In OCR-RCNN, $\mathbf{f}$ is generated by the region proposal network, which has low proposal accuracy and thus causes the bad performance of OCR-RCNN in character recognition. The outputs of the bounding box regressor are refinements

of the inaccurate region proposals [Fig. 2(a)] and can help generate more accurate $\mathbf{f}$. As a result, the recognition accuracy is significantly improved in OCR-RCNN-V2.

Moreover, another two important modifications are made to help improve inference efficiency.

- 1) As shown in Fig. 2, the original InceptionResnetV2 is replaced with a more efficient InceptionV2 [31] in OCR-RCNN-V2. Although this accelerates the inference speed, the model capacity is inevitably sacrificed to some extent.
- 2) To address this problem, another InceptionV2 is integrated into the recognition network, and some feature aggregation operations are also conducted.

As indicated by the bottom part of Fig. 2(b), we first crop button patches from the input image according to the bounding box regressor, and then these patches are processed by the OCR backbone to generate the corresponding image features $\mathbf{f}_r$. Similarly, according to the regressor, we can also locate button features $\mathbf{f}_s$ from the output of the detection backbone. Lastly, we aggregate the button features by concatenating $\mathbf{f}_r$ and $\mathbf{f}_s$, yielding a more representative feature $\mathbf{f}$ for the following character recognition.

2) **Loss Function:** Different from OCR-RCNN, which is an end-to-end trainable network, OCR-RCNN-V2 is trained in two stages. First, the detection network is trained following a standard procedure presented in [25]. Based on the outputs of the detection network, the recognition network is then trained using the following loss function:

$$L_r = \frac{1}{N} \sum_i \eta_i \sum_t \text{CCE}(\mathbf{o}_i^t, \mathbf{g}_i^t) \quad (5)$$

where CCE is the categorical cross entropy, N is the batch size, and η_i is the detection score of i th button candidate, indicating the probability of this candidate being identified as a real button. In OCR-RCNN, $\eta_i \in \{0, 1\}$ is an indicator used for removing illegal proposals generated by the region proposal network. To enlarge the data set, we use both the ground-truth and predicted bounding boxes to generate training samples. For ground-truth data, η is set to one. For low-quality predicted samples, η is with a tiny value and thus autonomously eliminates their disturbance to the network.

B. Button Pose Estimation

As shown in Fig. 3, the current frame $\mathcal{F}^c$ and the desired frame $\mathcal{F}^d$ give the current and desired poses of the camera, respectively. Button pose estimation aims to estimate the transformation between $\mathcal{F}^c$ and $\mathcal{F}^d$.

1) **Uncertainty Model:** Considering an image feature point $\mathbf{x}_i^d$, which is the projection of a 3-D point $\mathbf{p}_i^d$. If we measure $\mathbf{p}_i^d$ in $\mathcal{F}^c$, its position should be $\mathbf{p}_i^c = R \mathbf{p}_i^d + \mathbf{t}$, where $[R|\mathbf{t}] \in SE(3)$ represents the pose of $\mathcal{F}^d$ with respect to $\mathcal{F}^c$. Ideally, the projection of $\mathbf{p}_i^c$ on the current image plane should be $\mathbf{x}_i^c$. However, since the image processing and measurement of $\mathbf{x}_i^c$ are inevitably corrupted with noises, the actual projection of $\mathbf{x}_i^c$ may be distorted to $\mathbf{y}_i^c$. This thus introduces perception uncertainties, which can be modeled by a Gaussian distribution

$$\bar{\mathbf{y}}_i^c - \frac{1}{z_i} K (R \mathbf{p}_i^d + \mathbf{t}) \sim \mathcal{N}(\cdot | \mathbf{0}, \text{diag}([\sigma_u, \sigma_v, 0])) \quad (6)$$

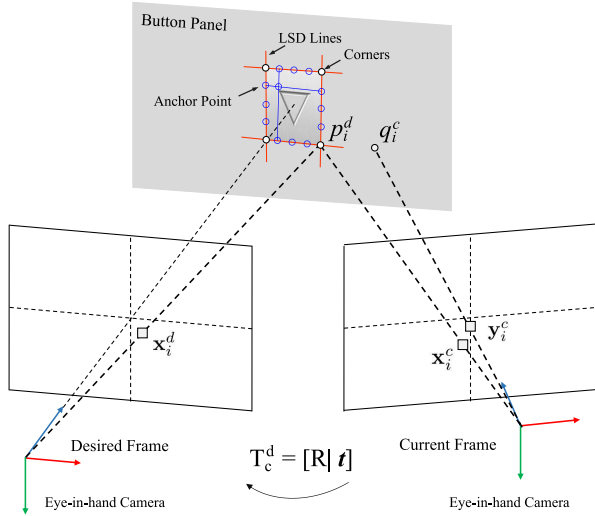


Fig. 3. Illustration of uncertainty modeling and point cloud sampling.

where z_i is the depth of p_i^c and the covariance is measured in image planes. Although (6) provides a straightforward explanation for the uncertainties, it is difficult to optimize, since z_i depends on the unknown variable $[R|t]$. To address this problem, we backproject y_i^c into $\mathcal{F}_c$ (q_i^c in Fig. 3) and model the uncertainties as follows:

$$q_i^c - (R p_i^d + t) \sim \mathcal{N}(\cdot | \mathbf{0}, \Sigma), \quad \Sigma = \text{diag}([\sigma_x, \sigma_y, \sigma_z]). \quad (7)$$

The uncertainties still follow a Gaussian distribution after the backprojection, but the orientation and magnitude of the covariance are changed [17]. The button pose estimation problem in this work is formulated as finding optimal $[R|t]$ to fit a least-uncertainty model defined in (7). To achieve this purpose, we also need to generate (p^d, q^c) and then estimate the pose by solving an MLE problem.

2) *Point Cloud Sampling*: As shown in Fig. 3, the point cloud sampling process includes four steps: 1) the intersections of LSD lines are taken as button corners; 2) some equally spaced anchor points are sampled at each of the button edges; 3) connect the anchors by pairs and take the intersections as point samples (the corner and anchor points are also included); and 4) backproject these point samples into the current camera frame $\mathcal{F}_c$ to generate q^c . In terms of p^d , it can be directly obtained by parameter settings of the desired pose. For example, suppose a button is with a size of 0.03 m, and it is required to be located at the front of the camera with a distance of 0.1 m and no rotations, then the coordinate of the bottom-right corner (Fig. 3) is $p_i^d = (0.015, 0.015, 0.1)$. Based on the corner points, we then sample the desired point cloud according to the second and third steps of generating q^c . The advantage of this method is that the correspondences between p^d and q^c can be directly obtained, thus avoiding the expensive point matching process. The disadvantage is that the sampling process may introduce large uncertainties to q^c .

3) *Pose Estimation*: Suppose N sampled points, the least-uncertainty model can be achieved by solving

the following MLE problem with respect to θ, t, Σ :

$$\begin{aligned} \mathcal{E} &= - \sum_{i=1}^N e_i^T \Sigma^{-1} e_i - N \ln |\Sigma|, \\ e_i &= q_i^c - (R(\theta) p_i^d + t) \end{aligned} \quad (8)$$

where $|\Sigma|$ denotes the determinant of Σ , $R(\theta)$ is Rodrigues' formula that relates rotation matrix R and axis-angle representation θ

$$R(\theta) = \mathbf{I}_{3 \times 3} + \frac{\sin(\|\theta\|)}{\|\theta\|} [\theta]_{\times} + \frac{1 - \cos(\|\theta\|)}{\|\theta\|^2} [\theta]_{\times}^2 \quad (9)$$

where $\|\theta\|$ denotes L1 norm of θ and $[\theta]_{\times}$ is a skew matrix

$$[\theta]_{\times} = \begin{bmatrix} 0 & -\theta_z & \theta_y \\ \theta_z & 0 & -\theta_x \\ -\theta_y & \theta_x & 0 \end{bmatrix}. \quad (10)$$

The problem defined in (8) does not have a closed-form solution, and thus the numerical optimization method is leveraged to solve it. Herein, we directly present the Jacobian functions, which can be used to calculate the Jacobian matrix during optimization

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial \theta_k} &= \text{Tr} \left[\left(\frac{\partial \mathcal{E}}{\partial R} \right)^T \frac{\partial R}{\partial \theta_k} \right] \\ \frac{\partial \mathcal{E}}{\partial R} &= 2 \sum_{i=1}^N \Sigma^{-1} e_i x_i^T \\ \frac{\partial \mathcal{E}}{\partial t} &= 2 \sum_{i=1}^N \Sigma^{-1} e_i \\ \frac{\partial \mathcal{E}}{\partial \Sigma} &= \sum_{i=1}^N \Sigma^{-1} e_i e_i^T \Sigma^{-1} - N \Sigma^{-1} \end{aligned} \quad (11)$$

where $\partial R / \partial \theta_k$ is given in the appendix of [32] and [33]. Note that if we let $\partial \mathcal{E} / \partial \Sigma = \mathbf{0}$, then $\Sigma = 1/N \sum_{i=1}^N e_i e_i^T$, which means Σ can be calculated using the estimated $[R|t]$ at each numerical iteration step, and thus only the first three functions are needed for calculating the Jacobian matrix.

C. Control Schema

1) *Position Controller*: Based on the estimated pose, a screw-based position controller is leveraged to operate the button. The estimated pose, $s = [\theta, t]^T$, is also called *twist* or *screw*, which represents a unit-time movement from $\mathcal{F}_c$ to $\mathcal{F}_d$ around a fixed axis $[\omega, v]^T$. The fixed axis is called *screw axis* and the movement distance is called *screw displacement* k , which can be calculated according to the following rules [19].

- 1) If $\theta = \mathbf{0}$, then $k = \|t\|$ and $[\omega, v]^T = [\mathbf{0}, t/k]^T$.
- 2) Otherwise, $k = \omega / \|\omega\|$ and $[\omega, v]^T = [\omega/k, G^{-1}t]^T$, where

$$G^{-1} = \frac{1}{k} I - \frac{1}{2} [\omega]_{\times} + \left(\frac{1}{k} - \frac{1}{2} \cot \frac{k}{2} \right) [\omega]_{\times}^2. \quad (12)$$

Here, the screw axis gives the desired movement direction of the eye-in-hand camera, and k gives the displacement along

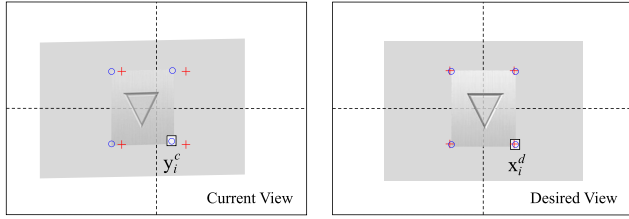


Fig. 4. Illustration of visual servo control. Red cross and blue circle are the desired and current positions of button corners, respectively.

that direction. Based on the screw representation of poses, an efficient control law is designed as follows:

$$\mathbf{u}_p = \gamma k[\boldsymbol{\omega}, \mathbf{v}]^T \quad (13)$$

which is essentially a P-controller in $se(3)$. As the control signal $\mathbf{u}_p$ is exerted on the camera, we thus also need to transform it into motor control signals by leveraging the inverse kinematics of the manipulator. The transformation follows a standard procedure [19], and we do not present it due to the limit of space. Limited to the positioning error of manipulator, eye-in-hand calibration error, and pose estimation error, the position controller cannot meet the accuracy requirement of button operation. We thus leverage a visual servo controller to tune the pose of the manipulator.

2) *Visual Servo Controller*: As shown in Fig. 4, the goal of visual servo control is to align the current corner points with the desired ones by changing camera poses in 3-D space, which can be achieved by minimizing the following position error:

$$\bar{\mathbf{e}}_i = K^{-1}(\bar{\mathbf{y}}_i^c - \bar{\mathbf{x}}_i^d) \quad (14)$$

where $K^{-1}\bar{\mathbf{y}}_i^c$ and $K^{-1}\bar{\mathbf{x}}_i^d$ are homogeneous coordinates of $\mathbf{y}_i^c$ and $\mathbf{x}_i^d$ in the normalized image plane, respectively. To ensure an exponential decoupled decrease of $\bar{\mathbf{e}}_i$ with respect to time t , the following ordinary differential equation should be satisfied:

$$\frac{\partial \bar{\mathbf{e}}_i}{\partial t} = -\lambda \bar{\mathbf{e}}_i. \quad (15)$$

As indicated in Fig. 4, $\mathbf{x}_i^d$ (the red cross) is a constant value

$$\frac{\partial \bar{\mathbf{e}}_i}{\partial t} = \frac{\partial K^{-1}\bar{\mathbf{y}}_i^c}{\partial t}. \quad (16)$$

Equation (16) defines the motion of a feature point in the normalized image plane, which is proved related to camera motion $\dot{\mathbf{s}}$ [16]

$$\frac{\partial \bar{\mathbf{e}}_i}{\partial t} = L_i \dot{\mathbf{s}} \quad (17)$$

where L_i is an *interaction matrix*

$$L_i = \begin{bmatrix} xy & -(1+x^2) & y & \frac{-1}{z} & 0 & \frac{x}{z} \\ 1+y^2 & -xy & -x & 0 & \frac{-1}{z} & \frac{y}{z} \end{bmatrix} \quad (18)$$

where $[x, y, 1]^T = K^{-1}\bar{\mathbf{y}}_i^c$, which is the coordinate of feature point $\mathbf{y}_i^c$ in the normalized image plane, and z is the depth of $\mathbf{y}_i^c$ measured by the RGBD sensor. In (17), the third dimension of $\partial \bar{\mathbf{e}}_i / \partial t$ is zero, we thus directly omit it in

Algorithm 1 Elevator Button Operation

Input: $s_0, \Sigma_0, \mathcal{I}$ ▷ $\mathcal{I}$ is a raw image
1: $\mathcal{B} \leftarrow \text{RecognizeButton}(\mathcal{I})$ ▷ $\mathcal{B}$ is a button list
2: $b_t \leftarrow \text{TrackTarget}(\mathcal{B})$ ▷ start to track b_t
3: **if** NotExist(b_t)
4: **return** Failed
5: **end if**
6: $\mathcal{C} \leftarrow \text{DetectCorners}(b_t)$
7: $\mathbf{X} \leftarrow \text{SamplePoints}(\mathcal{C}, b_t)$
8: $\mathbf{s}, \Sigma \leftarrow \text{EstimatePose}(\mathbf{X}, s_0, \Sigma_0, 5)$ ▷ Eq. (8)
9: ExecutePositionControl($\mathbf{s}$)
10: **while** $s > s_{tol}$ and $t < t_{tol}$ **do** ▷ tolerance
11: $\mathcal{C} \leftarrow \text{DetectCorners}(b_t)$
12: $\mathbf{X} \leftarrow \text{SamplePoints}(\mathcal{C}, b_t)$
13: $\mathbf{s} \leftarrow \text{EstimatePose}(\mathbf{X}, \mathbf{s}, \Sigma, 1)$ ▷ iterate once
14: $\mathbf{u}_p \leftarrow \text{ControlSignal}(\mathbf{s})$ ▷ Eq. (13)
15: ExecutePositionControl($\mathbf{u}$)
16: **end while**
17: **if** $s < s_{tol}$ ▷ accuracy tolerance is achieved
18: ExecuteVisualServoControl($\mathcal{I}$)
19: **return** Success
20: **else** ▷ time budget is used up
21: **return** Failed
22: **end if**

the calculation. Suppose there are M feature points and let $L = [L_1, L_2, \dots, L_M]^T \in \mathbb{R}^{2M \times 6}$ ($M > 3$ is required to ensure there is no singularity in L), camera motions that can exponentially decrease $\bar{\mathbf{e}}$ are derived as follows:

$$\dot{\mathbf{s}} = -L^+ \frac{\partial \bar{\mathbf{e}}}{\partial t} = -\lambda L^+ \bar{\mathbf{e}} \quad (19)$$

where $\bar{\mathbf{e}} = [\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2, \dots, \bar{\mathbf{e}}_M]^T \in \mathbb{R}^{2M \times 1}$ is an error vector with the third dimension of $\bar{\mathbf{e}}_i$ being removed, and L^+ is the pseudoinverse of L . By selecting an appropriate λ , the control signal can be obtained, $\mathbf{u}_v = \dot{\mathbf{s}}$.

3) *Button Operation*: Based on the two controllers and the perception algorithms, a complete button operation procedure is presented in Algorithm 1. Rows 1–5 perform button recognition and tracking tasks, which are accomplished in the recognition thread [Fig. 1(b)]. Rows 6–16 are leveraged to estimate button poses and conduct position control, which are accomplished in the pose estimation thread and execution thread, respectively. Once the accuracy tolerance or time limitation is achieved, e.g., the pose estimation thread reports a pose error of $[12^\circ, 0.04 \text{ m}]$ within 30 s and the error is not further reduced after three consecutive loops, the controller is autonomously switched to the visual servo controller. It is worth noting that the three threads are continuously executed at specific frequencies until the operation is finished.

V. EXPERIMENTS

In this section, we first quantitatively evaluate the performance of each component and then qualitatively verify the effectiveness of the entire system. The training and testing of the network are based on the Tensorflow framework with a

TABLE I

EVALUATION RESULTS OF DIFFERENT BUTTON RECOGNITION METHODS. THE AR, AP, AND CHARACTER RECOGNITION ACCURACY ARE CALCULATED USING THE METRICS PRESENTED IN [4] AND [11]. THE SIGN “—” REPRESENTS THE PERFORMANCE IS NOT REPORTED

No.	Model	Testing Scale			Detection			OCR	Efficiency
		Buttons	Characters	Distortions	AR	AP ^{0.5IoU}	F1	Accuracy	Time (s)
1	Klingbeil’s work [4]	686	15	N	0.862	0.881	0.875	0.884	—
2	OCR-RCNN [14]	686	15	N	1.000	1.000	1.000	0.816	1.232
3	OCR-RCNN-V2	686	15	N	1.000	0.998	0.999	0.965	0.529
4	OCR-RCNN [14]	686	47	N	0.993	1.000	0.996	0.737	1.268
5	OCR-RCNN-V2	686	47	N	0.997	0.999	0.998	0.913	0.539
6	Dong’s work [11]	60	—	Y	0.900	0.876	0.888	0.876	—
7	OCR-RCNN [14]	60	—	Y	1.000	1.000	1.000	0.850	2.300
8	OCR-RCNN-V2	60	—	Y	1.000	1.000	1.000	1.000	0.271
9	OCR-RCNN [14]	60	47	Y	1.000	1.000	1.000	0.850	2.244
10	OCR-RCNN-V2	60	47	Y	0.950	1.000	0.974	0.950	0.268
11	OCR-RCNN [14]	6869	47	Y	0.920	0.998	0.958	0.620	0.791
12	OCR-RCNN-V2	6869	47	Y	0.854	0.998	0.920	0.738	0.187

TABLE II

CHARACTER DICTIONARY. FUNCTION BUTTONS ARE ENCODED BY THE COMBINATION OF SPECIAL CHARACTERS, E.G., > < (CLOSE) AND <> (OPEN)

‘0’: 0	‘1’: 1	‘2’: 2	‘3’: 3	‘4’: 4	‘5’: 5
‘6’: 6	‘7’: 7	‘8’: 8	‘9’: 9	‘A’: 10	‘B’: 11
‘C’: 12	‘D’: 13	‘E’: 14	‘F’: 15	‘G’: 16	‘H’: 17
‘I’: 18	‘J’: 19	‘K’: 20	‘L’: 21	‘M’: 22	‘N’: 23
‘O’: 24	‘P’: 25	‘R’: 26	‘S’: 27	‘T’: 28	‘U’: 29
‘V’: 30	‘X’: 31	‘Z’: 32	‘<’: 33	‘>’: 34	‘(’: 35
‘)’: 36	‘\$’: 37	‘#’: 38	‘&’: 39	‘s’: 40	‘-’: 41
‘*’: 42	‘%’: 43	‘?’: 44	‘!’: 45	‘φ’: 46	

GTX-TITAN-X (12G) GPU. The implementation of the system, including model inference, pose estimation, and control, is based on Python-2.7 with an ASUS ROG GM501 laptop.

A. Button Recognition

The performance of button recognition is evaluated on three data sets. Detailed evaluation results are presented in Table I.

1) *First Data Set*: It is introduced in [4] and consists of 686 test buttons. Images in this data set are captured in favorable light conditions and without any distortion. The character dictionary used by this data set contains 15 characters, i.e., 1–9, G, L, B, open, close and alarm. Button categories that can be recognized are number buttons (combinations of 1–9), three letter buttons (G, L, B), and three function buttons (open, close, alarm). A comparison with the work of Klingbeil *et al.* [4] is shown in rows 1–3 of Table I. OCR-RCNN and OCR-RCNN-V2 both exhibit very high F1 score on the detection task, while OCR-RCNN-V2 shows the best character recognition accuracy and efficiency. In this experiment, buttons that are not supported by the dictionary (designed in [4]) are labeled with NAL and not considered by the recognition task. However, OCR-RCNN and OCR-RCNN-V2 support an even larger dictionary (Table II) and can recognize more button categories. Therefore, we additionally perform an experiment on this data set to

compare OCR-RCNN and OCR-RCNN-V2. The result is shown in rows 4 and 5 of Table I. We can see the recognition accuracy is increased from 73.7% to 91.3% (improved by 17.6%) by using OCR-RCNN-V2, which significantly demonstrates the advantages of this new architecture.

2) *Second Data Set*: It is introduced in [11] and consists of 60 test buttons. Since the authors adopt a classification-based approach, no character dictionary is designed for this data set. Instead, the authors divide the buttons into 15 classes, i.e., 0–9, open, close, alarm, others, and negatives. Compared with [4] and [14], the capacity of this method is very limited. We follow the same procedure as the first experiment. The comparison results with the work of Dong *et al.* [11] are presented in rows 6–8 of Table I, and it is indicated that OCR-RCNN-V2 has a much better performance than [11] in both detection and recognition tasks. The additional evaluation results are presented in rows 9 and 10 of Table I. As we can see, compared with OCR-RCNN, the detection performance of OCR-RCNN-V2 is slightly reduced, but the recognition accuracy is increased from 85.0% to 95.0% (improved by 10%) and the running speed is also accelerated by eight times.

3) *Third Data Set*: It is introduced in [14] and contains 6869 buttons in 365 classes. Images in this data set are captured in challenging light conditions and with various perspective distortions. A dictionary with 47 characters is designed for this data set. The comparison results with OCR-RCNN [14] are presented in rows 11 and 12 of Table I. As we can see, the character recognition accuracy of OCR-RCNN-V2 achieves a significant improvement, about 11.8%. This result again verifies our claim in Section IV-A that the accurate proposal of button candidates has a great influence on character recognition. Meanwhile, there is a slight drop in AR and F1 in terms of button detection,² which is caused by the simplification of the detection backbone (Fig. 2). However, if considering the gain of runtime efficiency,

²To make a fair comparison, we adopt the standard evaluation metrics presented in [4] and [11] rather than the COCO metrics used in [14].



Fig. 5. Demonstration of button recognition results. The recognized characters are visualized at the center of each detected button.

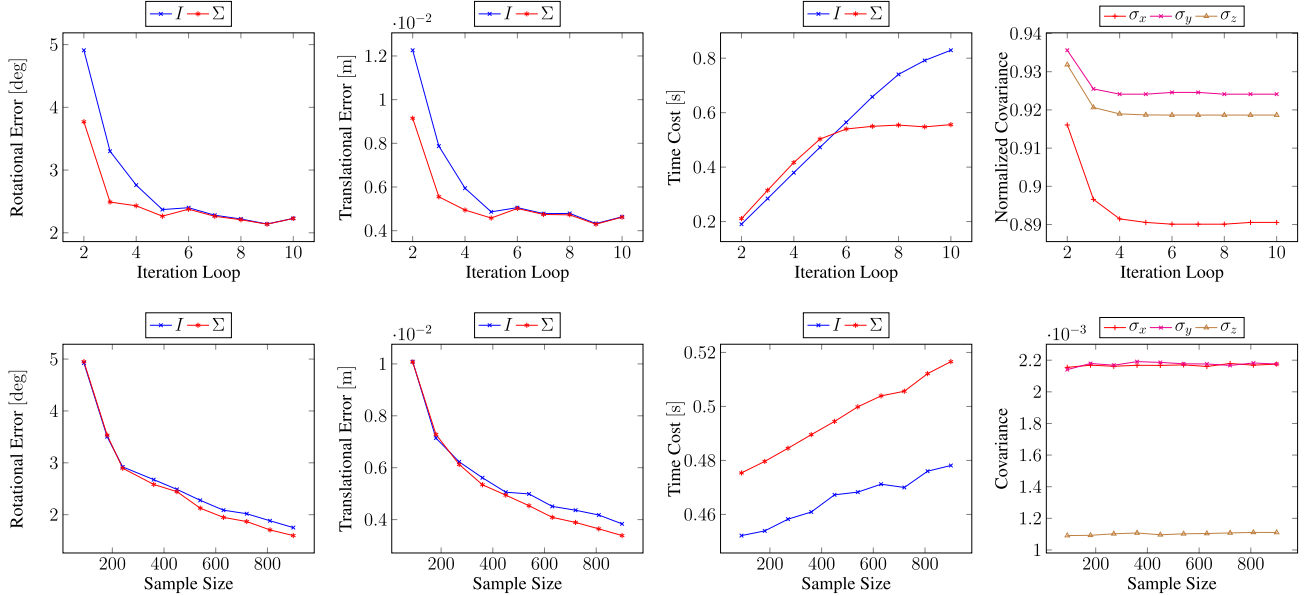


Fig. 6. Results of button pose estimation. Σ and I represent the cases that consider and not consider uncertainties, respectively.

which is improved by $4.2\times$, the slight drop in detection performance is acceptable. Furthermore, we conduct some model optimization and achieve a ~ 10 -Hz inference frequency ($640 * 480$ resolution) on a laptop [Fig. 1(a)]. As demonstrations, some evaluation results are visualized in Fig. 5. It can be seen that the proposed method can effectively address the recognition problem of different types of buttons.

B. Button Pose Estimation

The accuracy of pose estimation is mainly influenced by the number of iterations loops and sample size. We thus design two control experiments to evaluate our uncertainty model under different parameter settings.

1) *Iteration Loop*: We simulate a button point cloud $\mathbf{p}^d$ with 450 points and then apply a random transform to $\mathbf{p}^d$, yielding a new point cloud $\mathbf{q}^c$. Some noises with a covariance of $\sigma = [0.007, 0.007, 0.005]^T$ are added to $\mathbf{q}^c$ for simulating

the perception uncertainties. θ is bounded in $[-18.6^\circ, +18.6^\circ]$ and t is bounded in $[-0.1 \text{ m}, +0.1 \text{ m}]$ with $t_z \geq 0$. Based on the generated point clouds, we conduct experiments to compare our uncertainty model (denoted by Σ) with a baseline model that does not consider uncertainties (denoted by I). Nine different iteration loops are tested and each test repeats 200 times. The average performance is presented in the first row of Fig. 6. To better illustrate the changes of σ , the normalized values are used, and the actual values at the last test point are $[0.0022, 0.0022, 0.0011]^T$. We can see that our uncertainty model always shows better accuracy (the first two columns) and has a faster convergence speed (the last two columns). The results also show that at least five iteration loops are required to achieve an accurate pose estimation result.

2) *Sample Size*: We follow the same simulation procedure as the first experiment and set the iteration loop to 5. The sample size varies from 100 to 900 points. Each test repeats

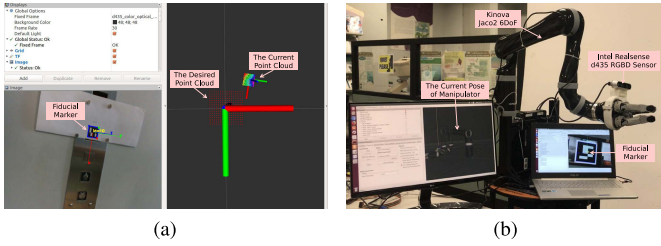


Fig. 7. Experiment settings for (a) pose estimation and (b) position control.

200 times. The average performance is shown in the second row of Fig. 6. As is indicated, when the sample size increases to about 250 points, the uncertainty model starts to show better estimation accuracy. When we further increase the sample size, the performance is also further improved. Larger point sets typically contain more information of button poses, hence the uncertainty model can be estimated more accurately. In terms of the runtime efficiency, the uncertainty model costs more computing time, as a more complicated Jacobian is required. The covariance in this experiment shows consistent performance and is at the same magnitude with ground truth, which shows that the variance of the noise is well modeled and a local minimum is obtained by the numerical optimizer.

3) *Real-World Experiment*: According to the simulation results, the loop number and sample size are set to 5 and 250 in this experiment, respectively. An RGBD video sequence [Fig. 7(a)] is used to evaluate the pose estimation algorithm. The ground truth is provided by a fiducial marker. By comparing the estimated pose with ground truth, the experiment achieves an average rotational and translational error of $(7.20^\circ, 0.025 \text{ m})$ with our uncertainty model, and $(9.17^\circ, 0.029 \text{ m})$ with the baseline model. The initial covariance is set to an identity matrix, and the final outcome is $\sigma = [1.6\text{e-}4, 1.5\text{e-}4, 4.5\text{e-}6]^T$. Note that the rotation and translation between the marker and the camera are very large, about $[-30^\circ, +30^\circ]$ and $[-0.4 \text{ m}, +0.4 \text{ m}]$, respectively. The large displacements make it difficult to find a better local minimum for (8). Moreover, the noises introduced by image processing are enlarged by the large displacements after backprojection, which further reduces the accuracy of button pose estimation. As we shall see, these are also the leading causes of the relatively low position control accuracy.

C. Elevator Button Operation

In this section, we first evaluate the performance of position control, and then report the accuracy of visual servo control with a real-world button operation experiment.

1) *Position Control*: The experiment setting of position control is shown in Fig. 7(b), in which a fiducial marker is used to simulate the target button and provide ground-truth poses. The initial pose of the manipulator is sampled from $[-30^\circ, +30^\circ]$ and $[-0.4 \text{ m}, +0.4 \text{ m}]$ randomly. The button is ensured to be well captured. For initial pose estimation (line 8 in Algorithm 1), five iteration loops are executed. For the following pose estimations (line 13 in Algorithm 1), only one iteration loop is executed, since pose changes between consecutive executions are

TABLE III
PERFORMANCE OF POSITION CONTROL. THE FIRST AND LAST TWO ROWS ARE SIMULATION AND REAL-WORLD EXPERIMENT RESULTS, RESPECTIVELY

Model	Rotational Error (degree)	Translational Error (mm)	Frequency (Hz)
$I + u_p$	1.8614	4.8731	10.46
$\Sigma + u_p$	1.6530	4.3256	9.38
$I + u_p$	10.8364	26.7062	10.30
$\Sigma + u_p$	9.2667	24.0435	9.61

very small. Once the tolerant time or accuracy is achieved, the position control is considered finished. Herein, if a pose estimation error of $[12^\circ, 0.04 \text{ m}]$ is achieved within 30 s and the error is not further reduced after three consecutive executions, we stop the controller and record the final pose error.

Based on the experimental settings, we first conduct a simulation experiment (repeating 200 times) in the Gazebo simulator, and then a real-world experiment (repeating 20 times) using a Kinova manipulator. The results are shown in Table III. It can be seen that the control accuracy is improved in both experiments when the uncertainty model is adopted for pose estimation. The simulation results exhibit a satisfying control accuracy, which demonstrates the effectiveness of the proposed position controller. However, the real-world experiment indicates a relatively low control accuracy, which is essentially caused by the noises in image processing and hand-eye calibration. Nevertheless, the position control still effectively reduces the initial pose errors and enables visual servo control.

2) *Button Operation*: In this experiment, we test the effectiveness of the entire system and report servo control accuracy. As demonstrated in Fig. 8, a complete button operation procedure is as follows. The position controller first drives the manipulator to the target button, and then the visual servo controller is launched to fine-tune the pose of the end-effector. After that, a constant offset is applied to help finish the button pressing. There are 20 tests performed for external buttons, among which 14 tests achieve servo goals with an average error of ten pixels (6.25 mm). In the successful cases, 11 tests finish button pressing, and three tests fail due to ineffective contact. Furthermore, we have also conducted 20 tests inside the elevator, shown in Fig. 9, and 16 tests successfully finish the operation task. Averagely, a success rate of 67.5% is achieved. Two demonstrations of visual servo control are visualized in Fig. 10, in which the goal is to align the small circles with red crosses. As is shown, the pose error (the first column) is corrected by the visual servo controller (the second column).

The failure case in this experiment is mainly caused by two reasons: 1) the poor light conditions, specular reflection, and image blur reduce the stability of button corner detection, which thus introduces some noises to visual servo control and 2) the end-effector of the manipulator has a large size [see Fig. 8(e)] and thus often gets stuck at button edges by a small control error. To solve these problems, we first apply a moving average filter to smooth the corner detection results, and then

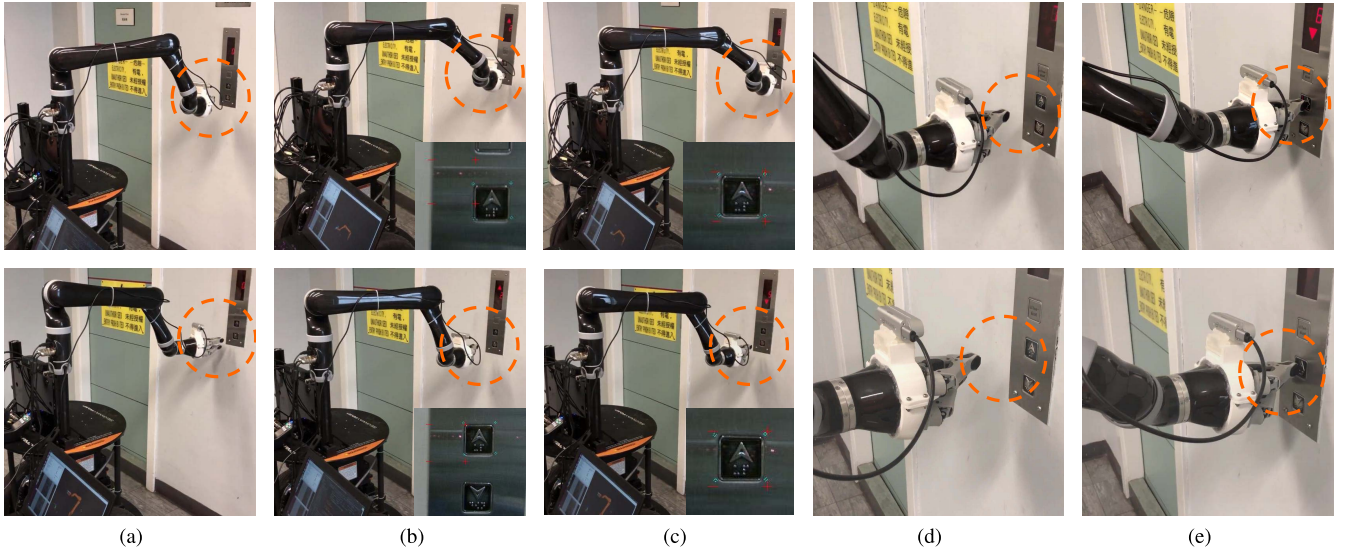


Fig. 8. Demonstrations of elevator button operation. The manipulator is placed close to the panel to ensure the button is well captured. (a) Start position control. (b) Start servo control. (c) Finish servo control. (d) Apply a constant offset. (e) Finish button operation.

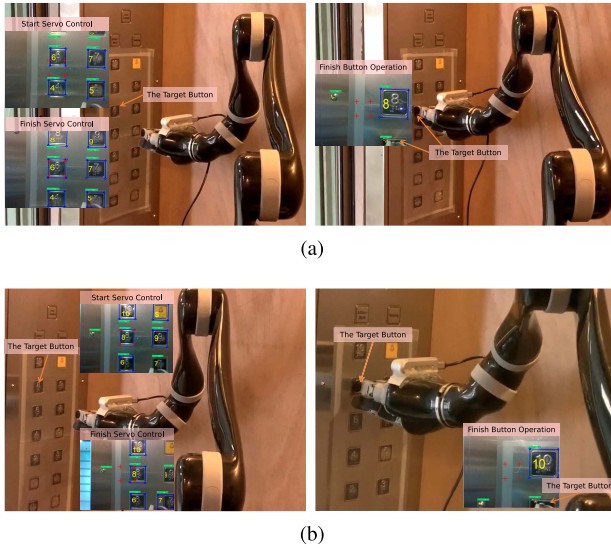


Fig. 9. Demonstration of button operation inside an elevator. (a) Button operation on sixth floor. (b) Button operation on eighth floor.

we mount a slim stick to the end-effector, as shown in Fig. 11, to increase the accuracy tolerance of button operation. Based on these improvements, we conduct another button operation experiment, and achieve a success rate of 95.0% (19/20) for external buttons and 90.0% (18/20) for internal buttons.

VI. DISCUSSION AND CONCLUSION

In this section, we first present our preliminary studies on the operation of circle buttons and then give a brief discussion about the operable region refinement. Finally, we conclude this article alongside some future work.

A. Operation of Circular Buttons

The difficulty of operating circular buttons lies in the estimation of their three-DoF orientations. As shown in Fig. 12(a), based on the point clouds of elevator panels, we can easily

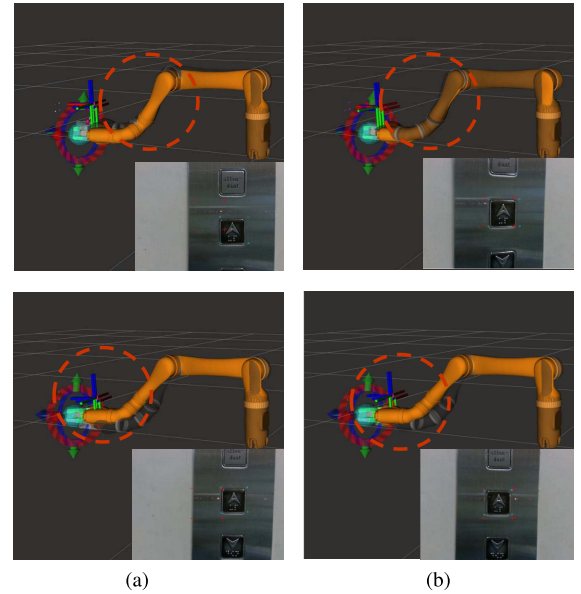


Fig. 10. Demonstration of the visual servo control process. (a) Start position control. (b) Start servo control.

fit a hyperplane for the buttons, which is represented by a surface normal. However, the surface normal can only provide two of three DoFs in the orientation, i.e., the rotation around the surface normal is not fixed. To constrain the third DoFs, we at least need to know the vertical or horizontal direction of the target elevator panel. Such information can be easily obtained by detecting the edges of rectangular buttons, but it cannot be achieved in terms of a circular button. Although the panel edges can be leveraged to estimate the vertical direction sometimes, they are not always observable during the entire operation process, and thus cannot be used as stable features. Therefore, we need to find some other constraints outside the space of image features.

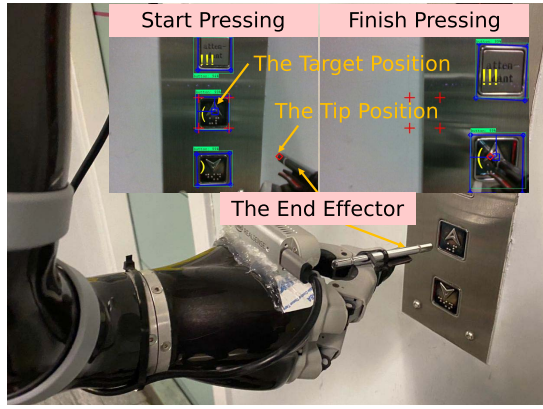


Fig. 11. Button operation using a slimmer end-effector.

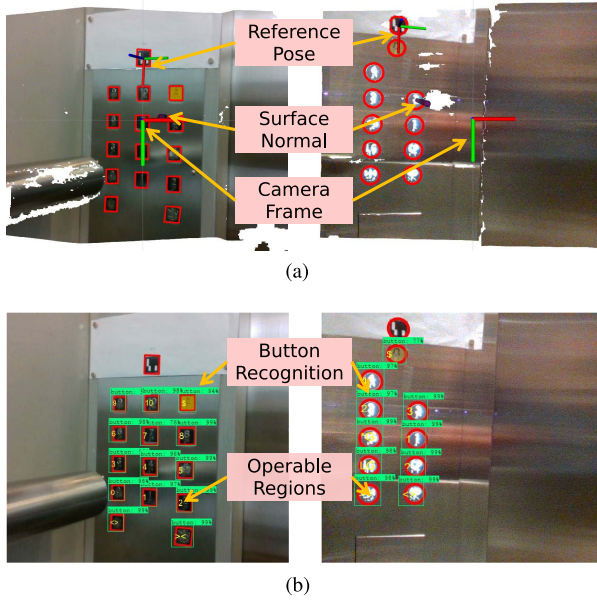


Fig. 12. Button pose estimation and operable region refinement. (a) Estimation of surface normal using the point clouds of button panels. (b) Operable region refinement (red color) on predicted bounding boxes.

As a supplementary algorithm for Section IV-B, we propose another pose estimation algorithm that can handle circular buttons. As shown in Fig. 13, we denote the base frame of the manipulator and the camera frame by o_b and o_c , respectively. The orientation of o_c with respect to o_b is denoted by c_bR , which can be obtained by forward kinematics of the eye-in-hand system. Since the y -axis of o_b (the green one) is always in a vertical direction, we can leverage it to constrain the rotation around the surface normal, and the method is as follows. We first specify a unit vector p_b aligned with the y -axis of o_b , and then transform it into o_c with $p_c = {}^c_bR^{-1}p_b$. The direction of p_c is then taken as the vertical direction of the elevator panel. Together with the surface normal p_n , we can also calculate the horizontal direction p_h , which is perpendicular to p_c and p_n , shown in Fig. 13. As a result, the orientation of the target button o_t with respect to o_c is obtained, denoted by t_cR . The translation t between o_t and o_c can be directly obtained from the button point cloud in Fig. 12(a). Based on the estimated button pose (${}^t_cR, t$), we then leverage Algorithm 1 and the slim end-effector

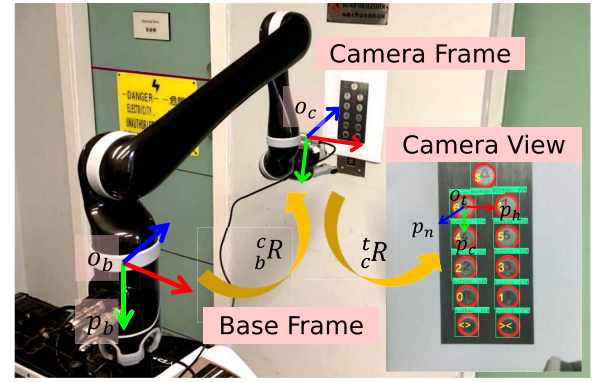


Fig. 13. Operation of circular buttons.

in Fig. 11 to conduct button operation. During visual servo control, the corners of a minimum enclosing rectangle of the target button serve as the control points. Based on our initial evaluation, the proposed framework can achieve a success rate of 95.5% (21/22 successful operations).

B. Operable Region Refinement

The detection outputs of the neural network are a set of bounding boxes. We thus need to refine the outputs to highlight the operable regions, shown in Fig. 12(b). Typically, the design of elevator panels follows specific patterns, which thus can be enumerated.

- 1) If the label and operable region share the same button (e.g., the first two columns in Fig. 5), or if they share the same button, but the operable region is an independent entity (e.g., the last column in Fig. 5), we first apply the ellipse and rectangle detection to crop the operable region from the predicted bounding box and then perform the button operation. The majority of modern elevator panels fall into these two categories.
- 2) If the label and operable region are located side by side and in different buttons (e.g., the third column in Fig. 5), the left is always the label and the right is always the operable region. Such a design is compatible with the reading habit of humans, and almost all manufacturers follow this pattern. Even if there occurs a counterexample, the robot can identify the operable button by performing a test with its force sensors.

Identifying these patterns is also not difficult. In our implementation, we first check whether there is a repetitive number button or an associated empty button around the target number button. If this is the case, we then take the right as the target. Otherwise, the detected number button is directly regarded as the target. After that, we then apply the circle and rectangle detection to refine the operable region.

C. Conclusion and Future Work

In this work, we present a systematic solution for autonomous elevator button operation. Three fundamental components are designed to address the problem of button recognition, pose estimation, and operation control, respectively. Extensive experiments show that our recognition

network can outperform state-of-the-art methods, and the pose estimation and control algorithms can effectively drive the manipulator to finish the button operation task. This is also the first work that presents a fully autonomous button operation system based on deep recognition networks. The proposed algorithms are compatible with different types of manipulators with an eye-in-hand configuration, and the presented system provides a vision-based solution that enables the robot to operate elevator buttons independently and without any external auxiliary, which significantly pushes forward the research on intelligent robot–elevator interaction.

One limitation of the current system is that the stability remains to be improved, which is also one of the focuses in our future work. Benefiting from the development of pose regression network, another promising solution is to construct an end-to-end neural network for simultaneous button detection, character recognition, and pose estimation. This may provide an even more general and robust solution for handling different types of elevator buttons. Some new sensor configurations can also be tried. For example, the eye-in-hand configuration can be replaced with an eye-to-hand configuration, which can help close the control loop and lead to better operation accuracy.

REFERENCES

- [1] T. Kanda, M. Shiomi, Z. Miyashita, H. Ishiguro, and N. Hagita, "An affective guide robot in a shopping mall," in *Proc. 4th ACM/IEEE Int. Conf. Hum. Robot Interact. - HRI*, 2009, pp. 173–180.
- [2] J. Palacin, J. A. Salse, I. Valganon, and X. Clua, "Building a mobile robot for a floor-cleaning operation in domestic environments," *IEEE Trans. Instrum. Meas.*, vol. 53, no. 5, pp. 1418–1424, Oct. 2004.
- [3] W. Gueaieb and M. S. Miah, "An intelligent mobile robot navigation technique using RFID technology," *IEEE Trans. Instrum. Meas.*, vol. 57, no. 9, pp. 1908–1917, Sep. 2008.
- [4] E. Klingbeil, B. Carpenter, O. Russakovsky, and A. Y. Ng, "Autonomous operation of novel elevators for robot navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2010, pp. 751–758.
- [5] J.-G. Kang, S.-Y. An, W.-S. Choi, and S.-Y. Oh, "Recognition and path planning strategy for autonomous navigation in the elevator environment," *Int. J. Control, Autom. Syst.*, vol. 8, no. 4, pp. 808–821, Aug. 2010.
- [6] H.-H. Kim, D.-J. Kim, and K.-H. Park, "Robust elevator button recognition in the presence of partial occlusion and clutter by specular reflections," *IEEE Trans. Ind. Electron.*, vol. 59, no. 3, pp. 1597–1611, Mar. 2012.
- [7] W. N. F. Wan Zakaria, M. R. Daud, S. Razali, and M. F. Abas, "Elevator's external button recognition and detection for vision-based system," *Proc. Electr. Eng. Comput. Sci. Informat.*, vol. 1, no. 1, pp. 265–269, Aug. 2014.
- [8] A. A. Abdulla, H. Liu, N. Stoll, and K. Thurow, "A robust method for elevator operation in semi-outdoor environment for mobile robot transportation system in life science laboratories," in *Proc. IEEE 20th Jubilee Int. Conf. Intell. Eng. Syst. (INES)*, Jun. 2016, pp. 45–50.
- [9] K. T. Islam, G. Mujtaba, R. G. Raj, and H. F. Nweke, "Elevator button and floor number recognition through hybrid image classification approach for navigation of service robot in buildings," in *Proc. Int. Conf. Eng. Technol. Technopreneurship (ICE2T)*, Sep. 2017, pp. 1–4.
- [10] D. Zhu, J. Liu, N. Ma, Z. Min, and M. Q.-H. Meng, "Autonomous removal of perspective distortion for robotic elevator button recognition," in *Proc. IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, Dec. 2019, pp. 913–917.
- [11] Z. Dong, D. Zhu, and M. Q.-H. Meng, "An autonomous elevator button recognition system based on convolutional neural networks," in *Proc. IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, Dec. 2017, pp. 2533–2539.
- [12] P.-Y. Yang, T.-H. Chang, Y.-H. Chang, and B.-F. Wu, "Intelligent mobile robot controller design for hotel room service with deep learning arm-based elevator manipulator," in *Proc. Int. Conf. Syst. Sci. Eng. (ICSSSE)*, Jun. 2018, pp. 1–6.
- [13] J. Liu and Y. Tian, "Recognizing elevator buttons and labels for blind navigation," in *Proc. IEEE 7th Annu. Int. Conf. CYBER Technol. Autom. Control, Intell. Syst. (CYBER)*, Jul. 2017, pp. 1236–1240.
- [14] D. Zhu, T. Li, D. Ho, T. Zhou, and M. Q. Meng, "A novel OCR-RCNN for elevator button recognition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 3626–3631.
- [15] A. Bazaei and M. Moallem, "Prestiction friction modeling and position control in an actuated rotary arm," *IEEE Trans. Instrum. Meas.*, vol. 59, no. 1, pp. 131–139, Jan. 2010.
- [16] F. Chaumette and S. Hutchinson, "Visual servo control. I. Basic approaches," *IEEE Robot. Autom. Mag.*, vol. 13, no. 4, pp. 82–90, Dec. 2006.
- [17] A. H. Abdul Hafez, S. Achar, and C. V. Jawahar, "Visual servoing based on Gaussian mixture models," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2008, pp. 3225–3230.
- [18] N. Marturi, B. Tamadazte, S. Dembele, and N. Piat, "Visual servoing-based depth-estimation technique for manipulation inside SEM," *IEEE Trans. Instrum. Meas.*, vol. 65, no. 8, pp. 1847–1855, Aug. 2016.
- [19] K. M. Lynch and F. C. Park, *Modern Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [20] B. Kuipers and A. Stroupe, "The AAAI-2002 robot challenge," *AI Mag.*, vol. 24, no. 1, pp. 65–76, Spring 2003. [Online]. Available: <https://aaai.org/Magazine/bibliography.php>
- [21] S. D. Science and T. A. (DSTA). (Aug. 2008). *Techx Challenge Rules*. [Online]. Available: <https://www.dsta.gov.sg/latest-news/news-releases/news-releases-2008/ab%out-the-techx-challenge>
- [22] T. Li, J. Pan, D. Zhu, and M. Q.-H. Meng, "Learning to interrupt: A hierarchical deep reinforcement learning framework for efficient exploration," in *Proc. IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, Dec. 2018, pp. 648–653.
- [23] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," 2016, *arXiv:1612.08242*. [Online]. Available: <https://arxiv.org/abs/1612.08242>
- [24] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.
- [25] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 91–99.
- [26] F. Wang, G. Chen, and K. Hauser, "Robot button pressing in human environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 7173–7180.
- [27] D. Zhu, T. Li, D. Ho, C. Wang, and M. Q.-H. Meng, "Deep reinforcement learning supervised autonomous exploration in office environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 7548–7555.
- [28] C. Wang, D. Zhu, T. Li, M. Q.-H. Meng, and C. W. de Silva, "Efficient autonomous robotic exploration with semantic road map in indoor environments," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2989–2996, Jul. 2019.
- [29] S. Chitta, I. Sucan, and S. Cousins, "MoveIt! [ROS topics]," *IEEE Robot. Autom. Mag.*, vol. 19, no. 1, pp. 18–19, Mar. 2012.
- [30] Z. Wojna et al., "Attention-based extraction of structured information from street view imagery," in *Proc. 14th IAPR Int. Conf. Document Anal. Recognit. (ICDAR)*, Nov. 2017, pp. 844–850.
- [31] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," 2015, *arXiv:1512.00567*. [Online]. Available: <https://arxiv.org/abs/1512.00567>
- [32] Z. Min, J. Wang, S. Song, and M. Q.-H. Meng, "Robust generalized point cloud registration with expectation maximization considering anisotropic positional uncertainties," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1290–1297.
- [33] Z. Min, D. Zhu, H. Ren, and M. Q.-H. Meng, "Feature-guided nonrigid 3-D point set registration framework for image-guided liver surgery: From isotropic positional noise to anisotropic positional noise," *IEEE Trans. Autom. Sci. Eng.*, early access, Jun. 23, 2020, doi: [10.1109/TASE.2020.3001207](https://doi.org/10.1109/TASE.2020.3001207).



Delong Zhu (Graduate Student Member, IEEE) received the B.S. degree in computer science and technology from Northeastern University, Shenyang, Liaoning, China, in 2015. He is currently pursuing the Ph.D. degree with the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong.

He spent nine months at the Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, as a Visiting Scholar. His research interest includes motion planning in dynamic environments and deep reinforcement learning.



Zhe Min received the B.S. degree in control science and engineering from Shandong University, Jinan, China, in 2014.

He is currently a Post-Doctoral Fellow at the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong. His current research interests include surgical navigation, medical robotics, and surface registration algorithm.

Mr. Min has served in the Program Committee in CIS-RAM 2017.



Tingguang Li received the B.E. degree in automation from Nanjing University, Nanjing, China, in 2016, and the Ph.D. degree from the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong, in 2020.

He was a Visiting Student Researcher at the Artificial Intelligence Laboratory, Stanford University, Stanford, CA, USA. His research interests include hierarchical reinforcement learning, autonomous exploration, and manipulation.



Max Q.-H. Meng (Fellow, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Victoria, Victoria, BC, Canada, in 1992.

He was the Director of the Advanced Robotics and Teleoperation (ART) Laboratory and a Professor with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB, Canada, for ten years. He has been a Professor of Electronic Engineering with The Chinese University of Hong Kong (CUHK), Hong Kong, since 2002. He

is currently a Professor and the Chairman of the Department of Electronic and Electrical Engineering, Southern University of Science and Technology, Shenzhen, China, on leave from the Department of Electronic Engineering, CUHK, and the Shenzhen Research Institute, CUHK, Shenzhen. He has authored over 500 journal and conference papers. His current research interests include robotics, perception and sensing, human-robot interaction, active medical devices, biosensors and sensor networks, and adaptive and intelligent systems.

Prof. Meng was a fellow of the Hong Kong Institution of Engineers and the Canadian Academy of Engineering. He has been an Elected Member of the Administrative Committee of the IEEE Robotics and Automation Society and has served on many editorial boards. He was a recipient of the IEEE Millennium Medal. He has served as the General Chair or the Program Chair for many international conferences, including the General Chair of IROS 2005 and IEEE ICRA 2021 to be held in Xi'an, China, in May 2021.



Tong Zhou received the B.S. degree in measurement, control technique, and instruments from the Harbin Institute of Technology, Harbin, Heilongjiang, China, in 2017. He is currently pursuing the Ph.D. degree with the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong.

His research interest includes robot motion planning in dynamic environments.